SSH Keys, SSH, SSL/TLS and OpenSSL

Concepts for the Linux Server Deployment Project

1. What are SSH keys?

An SSH key pair is made of two mathematically linked files:

- **Private key** (`id_ed25519`): stays on your own PC and is never shared.
- **Public key** (`id_ed25519.pub`): can be copied anywhere. It is placed on the server in `~/.ssh/authorized_keys`.

Analogy: the public key is a padlock installed on the server's door; the private key is the only key that opens it. Copies of the padlock can be given to anyone, but only you hold the key.

How the login works

1. The client connects and announces its username and public key.
2. The server checks that the key is listed in `authorized_keys`.
3. The server sends a random challenge.
4. The client signs the challenge with the private key (the key never leaves the PC).
5. The server verifies the signature with the public key. If it matches, access is granted.

Why it is better than a password

- A password can be guessed or brute-forced; a private key is a huge random number that cannot realistically be guessed.
- No secret is ever sent over the network.
- A stolen public key is useless to an attacker.
- With a passphrase, an attacker needs both the key file and the passphrase.

2. SSH vs SSL/TLS vs OpenSSL

These three are often confused because they use similar cryptography, but they are different things.

	SSH	SSL/TLS	OpenSSL
What it is	A protocol	A protocol	A software library and tool
Main purpose	Remote login, command execution, file transfer (SCP/SFTP)	Securing application traffic (HTTPS, email, APIs)	Implements SSL/TLS; generates keys and certificates
Typical port	22	443 (HTTPS), 993, 465...	None (not a protocol)
Typical use	Administering a server	A browser talking to a website	Creating certificates, testing TLS
Trust model	Keys you trust yourself (`known_hosts`, `authorized_keys`)	Certificates signed by a Certificate Authority (CA)	Produces the certificates used by TLS
Authenticates	The user (password or key)	Usually the server only	n/a

Key conceptual differences

- **Trust model:** with SSH you trust a server the first time you connect (the fingerprint question) and it is remembered. With TLS, trust comes from a CA (such as Let's Encrypt) that vouches for the website; browsers already trust these CAs.
- **Purpose:** SSH provides a secure *session* (a shell). TLS wraps *other* protocols: HTTP becomes HTTPS, SMTP becomes SMTPS.
- **Authentication:** SSH authenticates the *user*. TLS usually authenticates the *server*, and the user logs in later at the application level.

About the names

- **SSL** is the old name and is now deprecated and insecure. **TLS** is the modern version, though people still say "SSL".
- **OpenSSL** is a tool/library, not a protocol.
- **OpenSSH** is the software implementing SSH: `sshd` on the server, `ssh` on the client.

```
openssl s_client -connect google.com:443
```

The command above opens a TLS connection and shows the server certificate.

3. Quick commands

Generate a key pair (on the Windows PC, in PowerShell):

```
ssh-keygen -t ed25519
```

Copy the public key to the server:

```
type $env:USERPROFILE\.ssh\id_ed25519.pub | ssh asma@192.168.1.64 "mkdir -p ~/.ssh && chmod 700 ~/
```

Test that password login is refused after hardening (expected: `Permission denied (publickey)`):

```
ssh -o PubkeyAuthentication=no asma@192.168.1.64
```